The recursion principle of the circle: a point and a loop induce a map out of S^1 with a non-dependent computation rule

The Univalent Foundations Program

hott-book:hits.tex:local-snapshot-20261011; CC BY-SA 3.0 Unported

Editorial note. The counted statement is the author's 'thm:S1rec' (source lines 279-288, Lemma 6.2.5); the proof environment begins on the very next line, so the pairing is by adjacency.

Editorial note. The retained context is the author's own lead-in sentence, which is what makes the difficulty of the item visible: it announces that the recursion principle is derived from the induction principle and that the non-dependent computation rule is the tricky part.

Editorial note. The retained counter values (chapter 6, section 2, theorem offset 4) make the retained lemma print as Lemma 6.2.5, matching the original book numbering; theorem, lemma and corollary share one counter through aliascnt.

Editorial note. The two external references point at results of other chapters that the retained proof invokes by label; they are navigation pointers only and the counted proof is complete without them.

Editorial scope. One original result of the HoTT Book, counted once: the recursion principle of the circle with its non-dependent computation rule (Lemma 6.2.5).

Allowed standard prerequisites (editorial).

- Induction principle of the circle. For P : S^1 -> U with b : P(base) and a dependent path over loop from b to b there is f : prd x:S^1 P(x) with f(base) := b and apd f (loop) equal to the given path.. The whole proof is one application of this principle to the constant family lam x. A at source lines 290-291, and its output at lines 295-299 is stated with apd, not ap.
- Transport in a constant family. For a constant type family the transport along any path is the canonical identification transconst A p a : trans A p a = a.. It converts the datum p : a = a into the dependent path required by the induction principle at source lines 292-294, and it is the term that has to be cancelled at source line 303.
- Dependent paths. dpath B p u v unfolds to trans B p u = v, so a dependent path over loop from a to a is a path between transports of a.. The proof works throughout with the type dpath (x |-> A) loop a a and its relation to a = a, at source lines 291-294.

Source reference. Original lemma 2.3.5: sources/hott-hits/book-labelnumbers-first-edition.tex, source lines 1902–1902. Original result cited by the retained text but located outside the retained span. It is a navigation pointer: the counted statement's own author proof is retained line for line, and no part of it is replaced by this reference.

Source reference. Original lemma 2.3.8: sources/hott-hits/book-labelnumbers-first-edition.tex, source lines 1747–1747. Original result cited by the retained text but located outside the retained span. It is a navigation pointer: the counted statement's own author proof is retained line for line, and no part of it is replaced by this reference.

Of course, we expect to be able to prove the recursion principle from the induction principle, by taking P to be a constant type family. This is in fact the case, although deriving the non-dependent computation rule for loop (which refers to ap_f) from the dependent one (which refers to apd_f) is surprisingly a little tricky.

Lemma 6.2.5. *If A is a type together with $a : A$ and $p : a =_A a$, then there is a function $f : \mathbb{S}^1 \to A$ with*

$$f(\mathsf{base}) :\equiv a$$
$$\mathsf{ap}_f(\mathsf{loop}) \coloneqq p.$$

Proof. We would like to apply the induction principle of $\mathbb{S}^1$ to the constant type family, $(\lambda x.\, A) : \mathbb{S}^1 \to \mathcal{U}$. The required hypotheses for this are a point of $(\lambda x.\, A)(\mathsf{base}) \equiv A$, which we have (namely $a : A$), and a dependent path in $a =^{x \mapsto A}_{\mathsf{loop}} a$, or equivalently $\mathsf{transport}^{x \mapsto A}(\mathsf{loop}, a) = a$. This latter type is not the same as the type $a =_A a$ where p lives, but it is equivalent to it, because by original lemma 2.3.5 we have $\mathsf{transportconst}^A_{\mathsf{loop}}(a) : \mathsf{transport}^{x \mapsto A}(\mathsf{loop}, a) = a$. Thus, given $a : A$ and $p : a = a$, we can consider the composite

$$\mathsf{transportconst}^A_{\mathsf{loop}}(a) \cdot p : (a =^{x \mapsto A}_{\mathsf{loop}} a).$$

Applying the induction principle, we obtain $f : \mathbb{S}^1 \to A$ such that

$$f(\mathsf{base}) \equiv a \qquad \text{and} \tag{6.2.6}$$
$$\mathsf{apd}_f(\mathsf{loop}) = \mathsf{transportconst}^A_{\mathsf{loop}}(a) \cdot p. \tag{6.2.7}$$

It remains to derive the equality $\mathsf{ap}_f(\mathsf{loop}) = p$. However, by original lemma 2.3.8, we have

$$\mathsf{apd}_f(\mathsf{loop}) = \mathsf{transportconst}^A_{\mathsf{loop}}(f(\mathsf{base})) \cdot \mathsf{ap}_f(\mathsf{loop}).$$

Combining this with (6.2.7) and canceling the occurrences of $\mathsf{transportconst}$ (which are the same by (6.2.6)), we obtain $\mathsf{ap}_f(\mathsf{loop}) = p$. □